

자료구조 조사 및 구현 과제: 배열, 큐, 스택

1. 자료구조란?

자료구조(Data Structure)는 데이터를 효율적으로 저장하고 사용하기 위한 방식이다. 프로그램은 데이터를 다루기 위해 만들어진다. 이때 데이터를 아무렇게나 저장하면 추가, 삭제, 조회, 정렬, 검색 같은 작업이 비효율적이 될 수 있다. 자료구조는 다음과 같은 문제를 해결하기 위해 필요하다.

- 데이터를 어떤 순서로 저장할 것인가?
- 필요한 데이터를 얼마나 빠르게 찾을 수 있는가?
- 데이터를 추가하거나 삭제할 때 비용이 얼마나 드는가?
- 특정 상황에 가장 자연스러운 데이터 처리 방식은 무엇인가?

이번 과제에서는 가장 기본적인 자료구조인 **배열(Array)**, **큐(Queue)**, **스택(Stack)**을 정리한다.

2. 배열(Array)

2.1 배열의 정의

배열(Array)은 같은 종류의 데이터를 연속된 공간에 순서대로 저장하는 자료구조이다. 각 데이터는 인덱스(index)를 통해 접근한다.

인덱스:	0	1	2	3
값:	10	20	30	40

배열에서 첫 번째 데이터는 보통 인덱스 `0`에 저장된다.

2.2 배열의 동작 방식

배열은 인덱스를 이용해 데이터를 조회한다.

```
data = [10, 20, 30, 40]

print(data[0]) # 10
print(data[2]) # 30
```

출력 결과:

10
30

배열은 인덱스를 알고 있다면 원하는 위치의 데이터를 빠르게 조회할 수 있다. 하지만 중간에 데이터를 추가하거나 삭제하면 뒤쪽 데이터들의 위치를 옮겨야 하므로 비용이 커질 수 있다.

2.3 배열의 장점

- 인덱스를 이용해 특정 위치의 데이터를 빠르게 조회할 수 있다.
- 데이터가 순서대로 저장되어 반복 처리하기 쉽다.
- 구조가 단순해 이해하고 사용하기 쉽다.

2.4 배열의 단점

- 중간에 데이터를 추가하거나 삭제할 때 많은 데이터 이동이 발생할 수 있다.
- 일반적인 배열은 크기가 고정되어 있어 공간 관리가 필요하다.
- 같은 종류의 데이터를 저장하는 데 적합하다.

2.5 배열의 실제 활용 사례

- 학생 점수 목록 저장
- 상품 목록 저장
- 이미지 픽셀 데이터 저장
- 게임 캐릭터의 아이템 목록 저장
- 게시물 목록, 댓글 목록 관리

배열 활용 예제: 학생 점수 목록 관리

학생 점수처럼 순서가 있고 인덱스로 접근할 일이 많은 데이터는 배열로 관리하기 좋다.

아래 예제는 학생 이름과 점수를 같은 인덱스에 저장한 뒤, 특정 학생의 점수를 조회하고 평균 점수를 계산하는 코드이다.

```
students = ["김사과", "반하나", "오렌지", "이메론"]
scores = [85, 90, 78, 92]

def print_scores():
    # 배열은 인덱스를 기준으로 순서대로 데이터를 처리하기 좋다.
    for index in range(len(students)):
        print(f"{students[index]}: {scores[index]}점")

def get_score(student_name):
    # 이름 배열에서 학생의 위치를 찾고, 같은 인덱스의 점수를 조회한다.
    if student_name in students:
        index = students.index(student_name)
        return scores[index]
    return None

def get_average():
    # 점수 배열의 모든 값을 더해 평균을 계산한다.
    return sum(scores) / len(scores)

print_scores()
print("오렌지 점수:", get_score("오렌지"))
print("평균 점수:", get_average())
```

출력 결과:

```
김사과: 85점
반하나: 90점
오렌지: 78점
이메론: 92점
오렌지 점수: 78
평균 점수: 86.25
```

이 예제에서 배열은 학생 목록과 점수 목록을 순서대로 저장하는 데 사용된다. 같은 위치의 인덱스를 이용하면 학생과 점수를 연결해서 처리할 수 있다.

배열 활용 예제: 상품 목록에서 특정 상품 조회

쇼핑몰 상품 목록처럼 여러 데이터를 순서대로 보여줘야 하는 경우에도 배열을 사용할 수 있다.

아래 코드는 상품을 추가하고, 전체 상품을 출력하고, 사용자가 선택한 번호의 상품을 조회하는 흐름을 보여준다.

```
products = []

def add_product(name):
    # 배열의 마지막에 상품을 추가한다.
    products.append(name)

def show_products():
    # enumerate()를 사용해 인덱스와 상품명을 함께 출력한다.
    for index, product in enumerate(products):
        print(f"{index}번 상품: {product}")

def select_product(index):
    # 인덱스를 이용해 원하는 상품을 바로 조회한다.
    if 0 <= index < len(products):
        return products[index]
    return "존재하지 않는 상품"

add_product("노트북")
add_product("마우스")
add_product("키보드")
add_product("모니터")

show_products()

print("선택한 상품:", select_product(2))
print("잘못된 선택:", select_product(10))
```

출력 결과:

```
0번 상품: 노트북
1번 상품: 마우스
2번 상품: 키보드
3번 상품: 모니터
선택한 상품: 키보드
잘못된 선택: 존재하지 않는 상품
```

이 예제에서 배열은 상품을 순서대로 저장하고, 인덱스를 이용해 특정 상품을 조회하는 데 사용된다. 상품 목록처럼 순서와 위치가 중요한 데이터는 배열과 잘 어울린다.

2.6 배열 Python 구현 코드

파이썬의 리스트를 이용해 배열처럼 동작하는 클래스를 구현할 수 있다.

```
class MyArray:
    def __init__(self):
```

```

# 데이터를 저장할 내부 리스트
self.data = []

def append(self, value):
    # 배열의 맨 뒤에 값을 추가
    self.data.append(value)

def remove_at(self, index):
    # index 위치의 값을 삭제하기 전에 인덱스가 올바른지 확인
    self._check_index(index)

    # pop(index) 는 해당 위치의 값을 삭제하고, 삭제한 값 반환
    return self.data.pop(index)

def size(self):
    # 배열에 들어있는 데이터 개수 반환
    return len(self.data)

def _check_index(self, index):
    if index < 0 or index >= len(self.data):
        raise IndexError("인덱스 범위를 벗어났습니다.")

def __len__(self):
    # len(arr) 형태로 배열의 길이를 구할 수 있게 해준다.
    return len(self.data)

def __getitem__(self, index):
    # arr[index] 형태로 값을 조회할 수 있게 해준다.
    return self.data[index]

def __setitem__(self, index, value):
    # arr[index] = value 형태로 값을 수정할 수 있게 해준다.
    self._check_index(index)
    self.data[index] = value

def __str__(self):
    # print(arr) 를 했을 때 내부 리스트 형태로 출력되게 해준다.
    return str(self.data)

# 사용 예시
arr = MyArray()

# 배열에 값 추가
arr.append(10)
arr.append(20)
arr.append(30)

# 배열 전체 출력
print(arr)
# [10, 20, 30]

# 인덱스를 이용해 값 조회
print(arr[1])
# 20

# 인덱스를 이용해 값 수정
arr[1] = 99

```

```

print(arr)
# [10, 99, 30]

# len()으로 배열 길이 확인
print(len(arr))
# 3

# 특정 인덱스 값 삭제
deleted_value = arr.remove_at(0)
print(deleted_value)
# 10

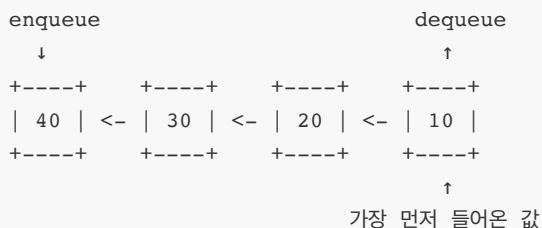
print(arr)
# [99, 30]

```

3. 큐(Queue)

3.1 큐의 정의

큐(Queue)는 먼저 들어온 데이터가 먼저 나가는 자료구조이다.
이 방식을 **FIFO(First In, First Out)**라고 한다.



동작 순서:

1. 10이 가장 먼저 들어온다.
2. 20, 30, 40은 10 뒤에 차례대로 들어온다.
3. dequeue를 하면 가장 먼저 들어온 10이 먼저 나간다.

3.2 큐의 동작 방식

큐는 데이터를 뒤쪽에 추가하고, 앞쪽에서 제거한다.

동작	설명
<code>enqueue()</code>	큐의 뒤쪽에 데이터를 추가한다.
<code>dequeue()</code>	큐의 앞쪽 데이터를 제거하고 반환한다.

3.3 큐의 장점

- 먼저 들어온 작업을 먼저 처리해야 하는 상황에 적합하다.
- 작업 순서를 공정하게 유지할 수 있다.
- 대기열 구조를 표현하기 쉽다.

3.4 큐의 단점

- 중간 데이터에 직접 접근하는 용도로는 적합하지 않다.
- 파이썬 리스트에서 앞쪽 데이터를 삭제하면 데이터 이동 비용이 발생할 수 있다.
- 큐의 목적은 순차 처리이므로 임의 조회가 필요한 경우에는 배열이 더 적합할 수 있다.

3.5 큐의 실제 활용 사례

- 프린터 출력 대기열
- 은행 번호표 대기 시스템
- 고객 상담 대기열
- 운영체제의 작업 스케줄링
- 메시지 큐
- BFS(너비 우선 탐색) 알고리즘

큐 활용 예제: 프린터 출력 대기열

프린터는 먼저 인쇄 요청을 보낸 문서부터 출력해야 한다. 따라서 FIFO 구조인 큐가 적합하다. 아래 코드는 문서를 대기열에 넣고, 먼저 들어온 문서부터 차례대로 출력하는 예제이다.

```
from collections import deque

print_queue = deque()

def request_print(document):
    # 새 인쇄 요청은 큐의 뒤쪽에 추가한다.
    print_queue.append(document)
    print(f"인쇄 요청 추가: {document}")

def print_next():
    # 가장 먼저 들어온 문서를 큐의 앞쪽에서 꺼낸다.
    if not print_queue:
        print("출력할 문서가 없다.")
        return

    document = print_queue.popleft()
    print(f"출력 완료: {document}")

request_print("문서1")
request_print("문서2")
request_print("문서3")

print("현재 대기열:", list(print_queue))

print_next()
print_next()

print("남은 대기열:", list(print_queue))
```

출력 결과:

```
인쇄 요청 추가: 문서1
인쇄 요청 추가: 문서2
인쇄 요청 추가: 문서3
현재 대기열: [ '문서1' , '문서2' , '문서3' ]
출력 완료: 문서1
출력 완료: 문서2
남은 대기열: [ '문서3' ]
```

이 예제에서 `request_print()` 는 `enqueue` 역할을 하고, `print_next()` 는 `dequeue` 역할을 한다. 먼저 요청한 문서가 먼저 출력되어야 하므로 큐가 적합하다.

큐 활용 예제: 고객 상담 대기열

고객 상담 시스템에서도 먼저 접수한 고객을 먼저 처리해야 공정하다.

아래 예제는 고객이 대기열에 등록되고, 상담사가 앞쪽 고객부터 처리하는 흐름을 나타낸다.

```
from collections import deque

waiting_customers = deque()

def register_customer(name):
    # 새 고객을 대기열의 뒤쪽에 추가한다.
    waiting_customers.append(name)
    print(f"{name} 접수 완료")

def serve_customer():
    # 가장 오래 기다린 고객을 먼저 상담한다.
    if not waiting_customers:
        print("대기 중인 고객이 없다.")
        return

    customer = waiting_customers.popleft()
    print(f"{customer} 상담 시작")

register_customer("고객A")
register_customer("고객B")
register_customer("고객C")
register_customer("고객D")

print("현재 대기 고객:", list(waiting_customers))

serve_customer()
serve_customer()

print("남은 고객:", list(waiting_customers))
```

출력 결과:

```
고객A 접수 완료
고객B 접수 완료
고객C 접수 완료
고객D 접수 완료
현재 대기 고객: ['고객A', '고객B', '고객C', '고객D']
고객A 상담 시작
고객B 상담 시작
남은 고객: ['고객C', '고객D']
```

큐는 접수 순서와 처리 순서가 같아야 하는 대기 시스템을 구현할 때 자연스럽다.

3.6 큐 Python 구현 코드

아래 코드는 큐의 기본 동작인 `enqueue()` 와 `dequeue()` 를 직접 구현한 예시이다.

```
class MyQueue:
    def __init__(self):
        # 데이터를 저장할 내부 리스트
        self.data = []

    def enqueue(self, value):
        # 큐의 맨 뒤에 값을 추가한다.
        self.data.append(value)

    def dequeue(self):
        # 큐가 비어 있으면 값을 꺼낼 수 없다.
        if self.is_empty():
            raise IndexError("큐가 비어 있습니다.")

        # 가장 먼저 들어온 값을 꺼낸다.
        # 리스트의 0번 인덱스가 큐의 앞쪽이다.
        return self.data.pop(0)

    def peek(self):
        # 큐가 비어 있으면 맨 앞 값을 확인할 수 없다.
        if self.is_empty():
            raise IndexError("큐가 비어 있습니다.")

        # 가장 앞에 있는 값을 삭제하지 않고 확인한다.
        return self.data[0]

    def is_empty(self):
        # 큐가 비어 있는지 확인한다.
        return len(self.data) == 0

    def size(self):
        # 큐에 들어있는 데이터 개수를 반환한다.
        return len(self.data)

    def __len__(self):
        # len(queue) 형태로 큐의 길이를 구할 수 있게 해준다.
        return len(self.data)

    def __str__(self):
        # print(queue)를 했을 때 내부 리스트 형태로 출력되게 해준다.
        return str(self.data)
```

```

# 사용 예시
queue = MyQueue()

# 큐에 값 추가
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print(queue)
# [10, 20, 30]

# 가장 앞에 있는 값 확인
print(queue.peek())
# 10

# 가장 먼저 들어온 값 꺼내기
print(queue.dequeue())
# 10

print(queue)
# [20, 30]

print(len(queue))
# 2

```

3.7 deque를 이용한 큐 구현

파이썬 리스트에서 `pop(0)` 은 앞쪽 데이터를 삭제한 뒤 나머지 데이터를 앞으로 당겨야 하므로 비효율적일 수 있다. 실제 큐 구현에는 `collections.deque` 를 사용하는 것이 더 좋다.

```

from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.items:
            print("큐가 비어 있다.")
            return None
        return self.items.popleft()

    def show(self):
        print(list(self.items))

queue = Queue()
queue.enqueue("A")
queue.enqueue("B")
queue.enqueue("C")

```

```
queue.show()
print(queue.dequeue())
queue.show()
```

출력 결과:

```
['A', 'B', 'C']
A
['B', 'C']
```

4. 스택(Stack)

4.1 스택의 정의

스택(Stack)은 나중에 들어온 데이터가 먼저 나가는 자료구조이다.
이 방식을 **LIFO(Last In, First Out)**라고 한다.

```
push pop
  ↓   ↑
+-----+
| 30 | <- 가장 나중에 들어온 데이터가 먼저 나간다.
+-----+
| 20 |
+-----+
| 10 |
+-----+
```

4.2 스택의 동작 방식

스택은 한쪽 끝에서만 데이터를 추가하고 제거한다.

동작	설명
<code>push()</code>	스택의 위쪽에 데이터를 추가한다.
<code>pop()</code>	스택의 위쪽 데이터를 제거하고 반환한다.

4.3 스택의 장점

- 가장 최근에 저장한 데이터를 빠르게 꺼낼 수 있다.
- 되돌리기, 뒤로가기처럼 이전 상태로 돌아가는 기능을 구현하기 쉽다.
- 구조가 단순하다.

4.4 스택의 단점

- 중간에 있는 데이터에 직접 접근하기 어렵다.
- 먼저 들어온 데이터를 먼저 처리해야 하는 상황에는 적합하지 않다.
- 너무 많은 데이터를 쌓으면 메모리 사용량이 커질 수 있다.

4.5 스택의 실제 활용 사례

- 웹 브라우저 뒤로가기

- 문서 편집기의 실행 취소
- 함수 호출 관리
- 괄호 검사
- DFS(깊이 우선 탐색) 알고리즘
- 계산기 수식 처리

스택 활용 예제: 웹 브라우저 뒤로가기

웹 브라우저는 사용자가 방문한 페이지를 순서대로 쌓아두고, 뒤로가기를 누르면 가장 최근 페이지부터 꺼낸다.
아래 코드는 현재 페이지를 관리하면서 뒤로가기 기능을 스택으로 표현한 예제이다.

```
back_stack = []
current_page = None

def visit(page):
    # 새 페이지로 이동하기 전에 현재 페이지를 뒤로가기 스택에 저장한다.
    global current_page

    if current_page is not None:
        back_stack.append(current_page)

    current_page = page
    print(f"현재 페이지: {current_page}")

def go_back():
    # 가장 최근에 방문했던 페이지로 돌아간다.
    global current_page

    if not back_stack:
        print("뒤로 갈 페이지가 없다.")
        return

    current_page = back_stack.pop()
    print(f"뒤로가기 후 현재 페이지: {current_page}")

visit("네이버")
visit("구글")
visit("유튜브")

print("뒤로가기 스택:", back_stack)

go_back()
go_back()
```

출력 결과:

```
현재 페이지: 네이버
현재 페이지: 구글
현재 페이지: 유튜브
뒤로가기 스택: ['네이버', '구글']
뒤로가기 후 현재 페이지: 구글
뒤로가기 후 현재 페이지: 네이버
```

가장 최근에 방문한 페이지를 먼저 꺼내야 하므로 스택의 LIFO 구조가 적합하다. `back_stack.pop()` 은 가장 마지막에 저장된 페이지를 꺼낸다.

스택 활용 예제: 괄호 검사

코드 편집기나 컴파일러는 괄호가 올바르게 열리고 닫혔는지 검사해야 한다. 이때 스택을 사용할 수 있다.

아래 코드는 `()`, `{}`, `[]` 세 종류의 괄호를 검사한다.

```
def is_valid_brackets(text):
    stack = []
    pairs = {
        ")": "(",
        "}": "{",
        "]" : "[",
    }

    for char in text:
        # 여는 괄호는 나중에 닫는 괄호와 비교하기 위해 스택에 저장한다.
        if char in "({[":
            stack.append(char)

        # 닫는 괄호를 만나면 가장 최근의 여는 괄호와 짝이 맞는지 확인한다.
        elif char in ")}]":
            if not stack:
                return False

            last_open = stack.pop()
            if last_open != pairs[char]:
                return False

        # 모든 검사가 끝났을 때 스택이 비어 있어야 올바른 괄호 구조이다.
        return not stack

print(is_valid_brackets("([])"))
print(is_valid_brackets("([)]"))
print(is_valid_brackets("((()))"))
```

출력 결과:

```
True
False
False
```

여는 괄호를 만나면 스택에 넣고, 닫는 괄호를 만나면 가장 최근에 들어온 여는 괄호를 꺼내 비교한다. 가장 최근에 열린 괄호가 가장 먼저 닫혀야 하므로 스택이 적합하다.

4.6 스택 Python 구현 코드

```
class MyStack:
    def __init__(self):
        # 데이터를 저장할 내부 리스트
        self.data = []

    def push(self, value):
        # 스택의 맨 위에 값을 추가한다.
        self.data.append(value)
```

```

def pop(self):
    # 스택이 비어 있으면 값을 꺼낼 수 없다.
    if self.is_empty():
        raise IndexError("스택이 비어 있습니다.")

    # 가장 마지막에 들어온 값을 꺼낸다.
    return self.data.pop()

def peek(self):
    # 스택이 비어 있으면 맨 위 값을 확인할 수 없다.
    if self.is_empty():
        raise IndexError("스택이 비어 있습니다.")

    # 가장 위에 있는 값을 삭제하지 않고 확인한다.
    return self.data[-1]

def is_empty(self):
    # 스택이 비어 있는지 확인한다.
    return len(self.data) == 0

def size(self):
    # 스택에 들어있는 데이터 개수를 반환한다.
    return len(self.data)

def __len__(self):
    # len(stack) 형태로 스택의 길이를 구할 수 있게 해준다.
    return len(self.data)

def __str__(self):
    # print(stack)을 했을 때 내부 리스트 형태로 출력되게 해준다.
    return str(self.data)

```

```

# 사용 예시
stack = MyStack()

```

```

# 스택에 값 추가
stack.push(10)
stack.push(20)
stack.push(30)

```

```

print(stack)
# [10, 20, 30]

```

```

# 가장 위에 있는 값 확인
print(stack.peek())
# 30

```

```

# 가장 마지막에 들어온 값 꺼내기
print(stack.pop())
# 30

```

```

print(stack)
# [10, 20]

```

```

print(len(stack))

```

출력 결과:

```
[ '페이지1' , '페이지2' , '페이지3' ]
뒤로가기: 페이지3
뒤로가기: 페이지2
[ '페이지1' ]
```

5. 배열, 큐, 스택의 차이점

배열, 큐, 스택은 모두 여러 데이터를 저장할 수 있지만 데이터를 꺼내는 방식이 다르다.

구분	배열(Array)	큐(Queue)	스택(Stack)
저장 방식	인덱스를 가진 순차 저장	들어온 순서대로 대기	위로 쌓는 구조
데이터 조회	인덱스로 직접 조회 가능	보통 앞쪽 데이터만 처리	보통 맨 위 데이터만 처리
데이터 제거	원하는 위치 삭제 가능	먼저 들어온 데이터 삭제	나중에 들어온 데이터 삭제
동작 원리	인덱스 기반	FIFO	LIFO
주요 연산	추가, 삭제, 조회	enqueue , dequeue	push , pop
활용 예시	목록, 점수표, 상품 리스트	대기열, 작업 처리	뒤로가기, 실행 취소

6. 추가 조사 과제 정리

6.1 배열과 Python 리스트는 완전히 같은 개념인가?

완전히 같은 개념은 아니다.

일반적인 배열(Array)은 같은 타입의 데이터를 연속된 메모리 공간에 저장하는 자료구조이다. 많은 언어에서 배열은 크기가 고정되어 있고, 같은 타입의 데이터만 저장한다.

반면 Python의 리스트(List)는 동적 배열(dynamic array)에 가까운 자료구조이다. 크기가 자동으로 늘어나고 줄어들며, 서로 다른 타입의 데이터도 저장할 수 있다.

```
data = [10, "Python", 3.14, True]
print(data)
```

출력 결과:

```
[10, 'Python', 3.14, True]
```

즉, Python 리스트는 배열처럼 인덱스로 접근할 수 있지만 일반적인 배열보다 더 유연한 자료구조이다.

구분	일반 배열	Python 리스트
크기	보통 고정	동적으로 변경 가능
타입	보통 같은 타입	여러 타입 저장 가능
인덱스 접근	가능	가능
내부 구조	연속된 메모리	동적 배열 기반 참조 저장

6.2 큐는 왜 FIFO 구조를 사용하는가?

큐는 먼저 들어온 작업을 먼저 처리해야 하는 상황을 표현하기 위해 만들어진 구조이다.
현실의 줄 서기와 비슷하다.

예를 들어 프린터에 여러 문서가 들어오면 먼저 요청한 문서가 먼저 출력되어야 공정하다. 은행 번호표도 먼저 온 사람이 먼저 처리되는 것이 자연스럽다.

따라서 큐는 순서를 공정하게 유지해야 하는 문제를 해결하기 위해 FIFO 구조를 사용한다.

6.3 스택은 왜 LIFO 구조를 사용하는가?

스택은 가장 최근에 저장한 데이터를 가장 먼저 꺼내야 하는 상황을 표현하기 위해 만들어진 구조이다.

예를 들어 웹 브라우저에서 페이지를 이동했다고 생각해보자.

방문 순서:
네이버 -> 구글 -> 유튜브

뒤로가기:
유튜브 -> 구글 -> 네이버

가장 최근에 방문한 페이지부터 되돌아가야 하므로 스택의 LIFO 구조가 적합하다.

6.4 웹 브라우저의 뒤로가기 기능은 어떤 자료구조를 사용할까?

웹 브라우저의 뒤로가기 기능은 스택(Stack)을 사용한다고 볼 수 있다.

페이지를 방문할 때마다 현재 페이지를 스택에 쌓고, 뒤로가기를 누르면 가장 최근 페이지를 꺼내 이전 페이지로 이동한다.

```
back_stack = []

back_stack.append("네이버")
back_stack.append("구글")
back_stack.append("유튜브")

print("현재 페이지:", back_stack.pop())
print("이전 페이지:", back_stack[-1])
```

출력 결과:

현재 페이지: 유튜브
이전 페이지: 구글

실제 브라우저는 뒤로가기뿐만 아니라 앞으로가기 기능도 제공하므로, 뒤로가기 스택과 앞으로가기 스택을 함께 사용할 수 있다.

7. 결론

배열, 큐, 스택은 모두 데이터를 저장하는 자료구조이지만 해결하려는 문제가 다르다.

- 배열은 인덱스로 데이터를 빠르게 조회하고 싶을 때 사용한다.
- 큐는 먼저 들어온 데이터를 먼저 처리해야 할 때 사용한다.
- 스택은 가장 나중에 들어온 데이터를 먼저 처리해야 할 때 사용한다.

자료구조를 공부할 때 중요한 것은 코드를 외우는 것이 아니라, **왜 이 구조가 필요한지**와 **어떤 문제를 해결하기 위해 만들어졌는지**를 이해하는 것이다.